

Operations in array (Search, Delete, and Insert)

➤ Search Operation

Searching is a fundamental concept in software development. Whether we're looking for a specific value in an array or trying to locate a record in a database, efficient searching algorithms are essential. In this lecture, we will explain two fundamental algorithms for searching data structures and algorithms:

- 1) **Linear Search:** A straightforward method that checks each element in an Unsorted array until the target is found, also known as Sequential Search, is the simplest searching algorithm. The search operation can be performed by linear traverses a list or array from the first element to the last element, comparing each element with the target value until a match is found or the list ends.



When to Use Linear Search?

- Unsorted Data: Linear Search doesn't require the data to be sorted.
- Small Data Sets: Efficient for small arrays where the overhead of more complex algorithms isn't justified.
- Simplicity: Easy to implement and understand, making it suitable for beginners.

Linear Search Algorithm in 1D Array

- 1- **Start** from the first element of the array.
- 2- Compare the **current** element with the **target** value.
- 3- If it **matches**, return the **index** of the element.
- 4- If it doesn't match, move to the **next** element.
- 5- Repeat steps 2–4 until the target is **found or the end** of the array is reached.

Linear Search Algorithm in 2D Array

1. First, we are declaring the array variable “**arr**” and the dimensions of the array with the first number for rows and second number for columns “ 2×3 ”.
2. We are then declaring two variables (i,j) for iterating over the elements in the array.
3. Then, for loops are used. The **outside** for loop is for the **rows** iteration and the **inside** loop is for the **columns**.
4. Create a variable named “**search**” to store the value of the element that need to be searched.
5. Create a variable named “**flag**” of bool type variable (the Boolean type is called **bool**) initialized with **false** value, that becomes **true** if the element is present in **arr**.
6. Run a **nested for loop** that will iterate over the **arr** and check if the current value is equal to the **search**’s value. If it is, then print its position and makes the value of **flag** equal to **true**, then break the loop.
7. Now, if flag becomes true then also break the outer loop.
8. After coming out from the nested loop, if $\text{flag} == 0$ then print “Not Found”.

Note:

To access diagonal elements or triangular halves, apply these condition on the row and column indexes

	$i < j$
	$i = j$
	$i > j$

$(i+j) < n$
$(i+j) = n$
$(i+j) > n$

$i \backslash j$	0	..	..	n
0				
:				
:				
n				

$i \backslash j$	0	..	..	n
0				
:				
:				
n				

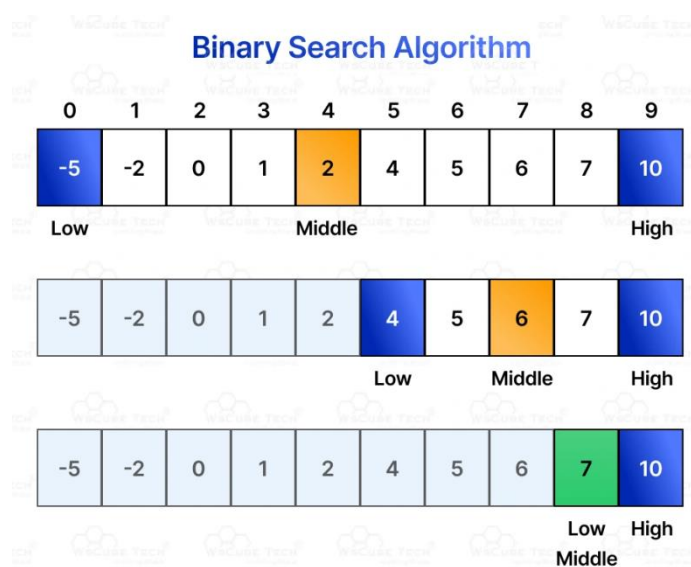
- 2) **Binary Search:** An efficient algorithm that repeatedly divides the search interval in half, requiring a sorted array. Binary Search is a highly efficient searching algorithm that works on sorted arrays. It repeatedly divides the search interval in half, eliminating half of the remaining elements each time, until the target value is found or the interval is empty.

When to Use Binary Search?

- **Sorted Data:** Binary Search requires the array to be sorted.
- **Large Data Sets:** Highly efficient for large arrays, reducing the number of comparisons significantly.
- **Performance-Critical Applications:** Ideal when performance is a priority, and the overhead of sorting (if needed) is manageable.

An Algorithm for Binary Search?

1. Prerequisite: The array must be **sorted** in ascending or descending order.
2. Determine the **middle** element of the array.
3. **Compare** the middle element with the target value.
 - If it matches, return the index.
 - If the target is smaller, repeat the search on the left half.
 - If the target is larger, repeat the search on the right half.
4. Continue the process until the target is found or the search interval is empty.



➤ Delete or Insert Operation

Deletion and Insertion are fundamental operations performed on arrays, allowing us to remove or add elements as needed. also used for managing and manipulating data within arrays. By understanding and implementing the deletion and Insertion algorithm, you can efficiently update and maintain arrays in your programs.

Why is Deletion and Insert Important?

- *Dynamic Management:* Allows for the dynamic management of data by adding wanted elements or removing unwanted elements.
- *Optimization:* Helps in optimizing the performance of applications by resize the data structure.

Types of Deletion and Insertion

- *At the Beginning:* Add or Remove the first element of the array.
- *At the End:* Add or Remove the last element of the array.
- *At a Specific Position:* Add or Remove an element from a given index in the array.

Algorithm for Deletion in an Array

The algorithm for deleting an element from an array involves the following **steps**:

1. *Start:* Initialize the process.
2. *Check Position:* Ensure the position is within the valid range of the array.
3. *Shift Elements:* Shift elements to the left to fill the gap created by the deleted element.
4. *Update Size:* Decrease the size of the array.
5. *End:* Print the updated array to verify the deletion and terminate the process.

Algorithm for Insert in an Array

The algorithm for deleting an element from an array involves the following **steps**:

1. **Start**: Initialize the process.
2. **Update Size**: increase the size of the array.
3. **Check Position**: Ensure the position is within the valid range of the array.
4. **Shift Elements**: Shift elements to the right to create empty location for the new element.
5. **Insert**: Insert the new element.
6. **End**: Print **the** updated array and terminate the process.

➤ Home work:

- Q1**: Write a Python program that uses **Linear Search** to find the value (2) and then delete it in a 1D array of 10 elements and return **True**, or return **False** if it is not found.
- Q2**: Write a Python program that uses **Linear Search** to find the value (3) in a **2D array (3,4)** and returns **TRUE** if it was existed, or returns **FALSE** if it is not found.
- Q3**: Write a Python program to **Insert** the value (5) in index (5) of a 1D array of 10 elements and return **TRUE** if successful, or return **FALSE** if not.
- Q4**: Write a Python program that uses **Binary Search** to find the value (7) in a 1D array 10 elements and returns **TRUE** if it was existed, or returns **FALSE** if it is not found.